

Competitive Exam Practice 12 — Answer Key

Instruction Pipelining and RISC vs. CISC

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 5, 2026

All eight items are [Original, GATE-pattern]; the local scanned PYQ source did not provide a complete, independently verifiable stem and answer for a Week 12 item. The calculations below are independent checks of the question file, especially the pipeline schedules and performance equation.

Q1 [Original, GATE-pattern] **10 cycles, speedup 3.75.** For $k = 5$ and $n = 12$,

$$T_{\text{pipe}} = k + n - 1 = 5 + 11 = 16, \quad T_{\text{seq}} = nk = 12 \times 5 = 60.$$

Thus $S = 60/16 = 3.75$. The limiting large-stream speedup is 5, but it is not the answer for this finite stream.

Q2 [Original, GATE-pattern] **cycle 8, 1 bubble.** The schedule is:

Instruction	1	2	3	4	5	6	7	8
LW	IF	ID	EX	MEM	WB			
ADD		IF	ID	stall	EX	MEM	WB	
SUB			IF	stall	ID	EX	MEM	WB

The load-use RAW dependence needs one bubble because the load value is not available for the immediately following EX stage. The ADD ALU result is forwarded to SUB, so no second bubble is needed.

Q3 [Original, GATE-pattern] **(B).** A single instruction still traverses IF, ID, EX, MEM, and WB, so its ideal latency is about five cycles. Once the pipeline is full, independent instructions can complete at one per cycle.

Q4 [Original, GATE-pattern] **0.06 CPI.** The expected penalty per instruction is

$$\Delta\text{CPI} = f_{br}p_{miss}P_{flush} = 0.20 \times 0.10 \times 3 = 0.06.$$

Q5 [Original, GATE-pattern] **(B).** Two stages demand one unavailable resource in the same cycle, which is a structural hazard. Separate instruction and data memories remove this particular conflict; stalling is another possible remedy.

Q6 [Original, GATE-pattern] **(A), (B), and (D).** These are design tendencies and a valid measurement rule. Statement (C) is too strong: instruction count, CPI, clock rate, stalls, cache behavior, code size, and the workload interact through $T_{CPU} = \text{IC} \times \text{CPI}/f$.

Q7 [*Original, GATE-pattern*] **(B)**. Using $T_{CPU} = IC \times CPI/f$,

$$T_R = \frac{1.20 \times 10^9 \times 1.1}{2.4 \times 10^9} = 0.550 \text{ s}, \quad T_C = \frac{0.80 \times 10^9 \times 2.0}{3.0 \times 10^9} = 0.533\bar{3} \text{ s}.$$

The CISC-like implementation is faster for this stated workload despite its higher CPI.

Q8 [*Original, GATE-pattern*] **(A), (B), and (C)**. A single instruction over many data elements is SIMD-style; independent cores running independent instruction streams are MIMD-style. A large working set can expose cache and virtual-memory costs. Statement (D) is false because total time also depends on instruction count, CPI, stalls, and memory behavior.

Source Boundaries and Verification

The questions are derived from the completed Week 12 deck and notes. The textbook concepts are standard treatments supported by the indexed sources: Patterson and Hennessy, ARM 5e, Ch. 4 §§4.5–4.8 (pipeline and hazards), Ch. 6 §§6.2–6.3 (parallelism), and App. D (RISC survey); Stallings 11e, Ch. 16 §16.4 and Ch. 17 §§17.5, 17.9 (pipeline and RISC/CISC). These are supporting references, not claims that any constructed question appeared in those books or in GATE. The local PYQ registry identifies Week 12 pages, but OCR did not recover a complete stem and answer; therefore no real PYQ claim is made here. All eight answers were recomputed from the explicit assumptions in the stems, including the two timing tables and both CPI/time calculations.